Estudo de Caso — Ingestão

Caso de Uso — ShopVibe: Ingestão de Dados em Tempo Real

Vencimento	Pontos	Entrega	Disponível
23 Out às 23:59	10	Caixa de entrada de texto ou URL de site	30 Set às 19:30 – 23 Out às 23:59

1. Visão geral do exercício

Você foi contratado como arquiteto(a) de dados da ShopVibe, um e-commerce fictício que perde vendas por não enxergar o que acontece na loja em tempo real. Sua missão é propor, documentar e demonstrar um pipeline que capture eventos da loja no momento em que acontecem.

Formato: grupos de 2 a 3 alunos.

O que será entregue

- Um ADR (Architectural Decision Record) justificando as tecnologias escolhidas.
- Um desenho da solução (diagrama de arquitetura).
- Uma demonstração prática mostrando eventos saindo da origem e chegando ao destino em tempo real.

Objetivos de aprendizagem

- Traduzir um problema de negócio em requisitos técnicos.
- Comparar tecnologias de streaming vistas no curso e justificar escolhas.
- Montar um ambiente com Docker e colocar um pipeline de ingestão para rodar.
- Documentar decisões de arquitetura no formato usado pelo mercado.

Regra principal: vocês podem usar qualquer tecnologia aprendida no curso, desde que o pipeline mostre dados fluindo em tempo real. A Seção 5 traz uma arquitetura de referência e a Seção 6 detalha cada tecnologia; usem como ponto de partida, não como resposta obrigatória.

2. A empresa e o problema de negócio

ShopVibe é um e-commerce brasileiro de moda e acessórios, fundado em 2019 em São Paulo. Hoje tem cerca de 1,2 milhão de clientes cadastrados, 15 mil pedidos por dia e picos de 20 vezes esse volume na Black Friday. Os números são fictícios e servem para dimensionar a solução.

Como a ShopVibe funciona hoje (AsIs)

- O site e o app gravam pedidos em um banco relacional (PostgreSQL).
- Todo dia, às 2h da manhã, um script copia as tabelas para uma planilha e para o time de BI.
- O estoque é atualizado por um sistema separado, também em lote, a cada 4 horas.

O que está dando errado

Na última Black Friday, a diretora comercial, Mariana Costa, relatou três problemas:

- **Venda de produto sem estoque:** 300 pedidos foram cancelados porque o site mostrava estoque que já tinha acabado. Cada cancelamento gerou reclamação e cupom de compensação.
- **Fraude descoberta tarde:** um grupo usou cartões clonados em dezenas de compras seguidas. O time só percebeu no relatório do dia seguinte, com R$ 180 mil em chargebacks.

- **Carrinhos abandonados sem reação:** 68% dos carrinhos foram abandonados. O time de marketing só conseguia disparar e-mail de recuperação no outro dia, quando o cliente já tinha comprado no concorrente.

O pedido da diretoria

> *"Quero saber o que está acontecendo na loja agora, não amanhã. Preciso de um painel ao vivo de vendas, alerta de pedido suspeito em segundos e estoque confiável no site."*

O desafio de vocês: desenhar e demonstrar a primeira versão (MVP) de uma plataforma de dados em tempo real para a ShopVibe. O foco do MVP é a captura e ingestão em tempo real dos eventos da loja; análises sofisticadas ficam como evolução.

3. Requisitos

O MVP precisa capturar pelo menos pedidos em tempo real; os demais eventos contam como diferencial.

Eventos da loja

Evento	Origem	Exemplo de campos	Obrigatório no MVP
Pedido criado	PostgreSQL (tabela orders)	order_id, customer_id, total, status, created_at	Sim
Item do pedido	PostgreSQL (tabela order_items)	order_id, product_id, quantity, price	Diferencial
Carrinho (add/remove)	App e site (API)	session_id, customer_id, product_id, action	Diferencial

Requisitos funcionais

- **RF01** — Capturar cada novo pedido e disponibilizá-lo para consumo em poucos segundos.
- **RF02** — Armazenar os eventos brutos para reprocessamento (camada raw / bronze).
- **RF03** — Manter um contador de vendas do dia (total em R$ e quantidade de pedidos) atualizado em tempo real (camada refined/gold).
- **RF04** — Expor os dados por uma API (ex.: GET /vendas/hoje, GET /pedidos/{id}).
- **RF05 (diferencial)** — Gerar alerta quando um mesmo cliente fizer mais de 5 pedidos em 2 minutos (regra simples de fraude).

Requisitos não funcionais

- **RNF01** — Latência: evento da origem ao destino em poucos segundos.
- **RNF02** — Escalabilidade: suportar 20 vezes o volume normal na Black Friday (cerca de 300 mil pedidos/dia).
- **RNF03** — Durabilidade: nenhum pedido pode ser perdido se um componente cair.
- **RNF04** — Reprodutibilidade: o ambiente inteiro sobe com um comando (docker compose up).
- **RNF05** — Custo: usar apenas ferramentas open source.

4. O que é um ADR (e um exemplo pronto)

Um ADR — Architectural Decision Record é um documento curto (1 a 2 páginas) que registra uma decisão de arquitetura: o contexto, o que foi decidido, quais alternativas foram descartadas e quais as consequências. O formato foi popularizado por Michael Nygard em 2011 e hoje é usado

em empresas como ThoughtWorks, Spotify e AWS.

Por que as empresas usam ADR? Daqui a um ano, alguém vai perguntar “por que escolhemos Kafka e não RabbitMQ?”. Sem ADR, a resposta se perde quando a pessoa sai da empresa. Com ADR, a justificativa está escrita e versionada junto com o código (normalmente em uma pasta docs/adr/ no Git).

Regras de ouro

- **Um ADR = uma decisão.** O grupo deve escrever pelo menos 3 ADR próprios (ex.: banco de destino ou forma de captura).
- **Numere e date:** ADR-001, ADR-002...
- **ADR não se apaga.** Se a decisão mudar, cria-se um novo ADR com status “Substitui ADR-00X”.
- **Status possíveis:** Proposto → Aceito → Obsoleto / Substituído.
- **Seja honesto nas consequências:** toda decisão tem pontos negativos. Um ADR só com vantagens perde nota.

Template (copie para o seu trabalho)

```
# ADR-NNN: <Título curto da decisão>

- Status: Proposto | Aceito | Substituído por ADR-XXX
- Data: AAAA-MM-DD
- Autores: <nomes do grupo>

## Contexto
Qual problema estamos resolvendo? Quais restrições e requisitos (RF/RNF) influenciam?

## Decisão
O que decidimos, em uma ou duas frases diretas.

## Alternativas consideradas
| Opção | Prós | Contras |
|---|---|---|
| Opção A | ... | ... |
| Opção B | ... | ... |

## Consequências
- Positivas: ...
- Negativas / riscos: ...
- O que muda para o time: ...
```

Exemplo completo: ADR-001

```
# ADR-001: Usar Apache Kafka como barramento de eventos em tempo real

- Status: Aceito
- Data: 2026-10-06
- Autores: Grupo 3 (Ana, Bruno, Carla, Diego)

## Contexto
A ShopVibe grava pedidos em PostgreSQL e só processa os dados uma vez por dia,
às 2h. Isso causou venda sem estoque, fraude detectada tarde e perda de carrinhos
na Black Friday. Precisamos que cada pedido chegue aos consumidores em menos de
5 segundos (RNF01), sem perda de dados se um componente cair (RNF03), e que
o mesmo evento possa ser lido por vários sistemas: painel, antifraude e estoque.
O volume de pico estimado é de 300 mil pedidos/dia (RNF02).

## Decisão
Adotar o Apache Kafka (modo KRaft, sem Zookeeper) como barramento central de
eventos. Cada tipo de evento terá seu próprio tópico (ex.: shopvibe.orders).
```

Alternativas consideradas (ADR-001)

Opção	Prós	Contras
Apache Kafka	Guarda eventos em disco e permite reprocessar; vários consumidores leem o mesmo tópico; ecossistema Kafka Connect/Debezium	Mais complexo de operar; consome mais memória
RabbitMQ	Simples de configurar; ótimo para filas de tarefas	Mensagem some depois de consumida; reprocessar histórico é difícil
Redis Streams	Muito rápido; já usamos Redis para cache	Dados em memória; retenção longa fica cara; ecossistema de conectores menor
Polling no PostgreSQL (SELECT a cada 5s)	Nenhuma ferramenta nova	Sobrecarrega o banco de produção; perde updates e deletes; não escala

```
## Consequências
- Positivas: vários consumidores independentes leem o mesmo evento; eventos
  ficam retidos por 7 dias e podem ser reprocessados; integração nativa com
  Debezium para captura de mudanças (CDC).
- Negativas / riscos: o time precisa aprender Kafka; o ambiente local exige
  pelo menos 4 GB de RAM a mais; tópicos e partições precisam ser planejados.
- O que muda: todo sistema novo publica eventos no Kafka em vez de escrever
  direto em outro banco. Próxima decisão: ADR-002 (como capturar os dados do
  PostgreSQL).
```

Ideias de ADRs para o grupo: ADR-002 — CDC com Debezium vs. producer na aplicação; ADR-003 — banco de destino (MongoDB, Cassandra, Elasticsearch…); ADR-004 — onde guardar o contador de vendas em tempo real (Redis?); ADR-005 — como entregar os dados (FastAPI).

5. Desenho da solução

O desenho da solução é criado pelo próprio grupo: ele deve atender todos os requisitos obrigatórios usando tecnologias do curso. Em uma arquitetura orientada a eventos, tudo o que acontece na loja vira evento em um tópico, e cada consumidor lê no seu ritmo, podendo escalar suas leituras.

O que é um desenho de solução

É um diagrama que mostra, em uma única página, por onde o dado nasce, como é capturado, onde é processado, onde fica guardado e como chega a quem vai usá-lo. Ele serve para alinhar o time técnico e o negócio: a diretora da ShopVibe precisa entender o fluxo sem ler código, e o desenvolvedor precisa ver quais componentes conversam entre si.

Camadas que o desenho precisa mostrar

Camada	O que representa	Perguntas que o desenho deve responder
Origem	Onde o evento nasce (banco transacional, app, site, sistema de estoque)	Quais eventos existem? Quem os gera?
Ingestão	Como o evento sai da origem e entra no pipeline	A captura é por CDC (leitura do log do banco) ou a aplicação publica o evento? Como garantir que nada se perde?

Camada	O que representa	Perguntas que o desenho deve responder
Transporte / streaming	O canal que distribui os eventos	Como vários consumidores leem o mesmo evento? Como escalar a leitura em picos como a Black Friday?
Processamento	Onde o evento é transformado, enriquecido ou avaliado	Onde são calculadas as métricas? Onde roda a regra de fraude?
Armazenamento	Onde o dado fica guardado, e para qual uso	Qual banco para cada necessidade: consulta por chave, contador ao vivo, busca, histórico bruto?
Entrega	Como o dado chega a quem consome	Painel, API, alerta? Qual a latência esperada?

Conceitos que ajudam a desenhar

- **Produtor e consumidor:** quem escreve e quem lê eventos. Um mesmo evento pode ter vários consumidores independentes.
- **Desacoplamento:** a origem não precisa saber quem vai usar o dado. Isso permite adicionar consumidores sem mexer no sistema de pedidos.
- **Escala horizontal:** a leitura é dividida entre várias instâncias do consumidor (ex.: partições e consumer groups no Kafka).
- **Camada bruta (raw/bronze):** guardar o evento original, sem transformação, permite reprocessar o histórico se uma regra mudar.
- **Persistência poliglota:** usar bancos diferentes para necessidades diferentes, em vez de um único banco para tudo.

Checklist do desenho

☐ Todas as camadas acima aparecem, com o nome da tecnologia escolhida em cada componente.

☐ As setas indicam o sentido da busca do dado e têm um rótulo curto (ex.: “publica”, “consome”, “grava”).

☐ Fica claro onde está o ponto de tempo real (onde o evento é capturado e onde é consumido).

☐ Cada escolha de tecnologia importante tem um ADR correspondente.

☐ Tem título e legenda, se cores ou formas tiverem significado.

Ferramentas sugeridas: draw.io (diagrams.net), Excalidraw, Miro ou papel e caneta fotografado.

Observações gerais

Importante:

- O prazo de entrega é impreterivelmente até às 23:59 do dia 23/10. Submissões realizadas após esse horário serão desconsideradas, independentemente do motivo alegado.
- O não cumprimento das instruções descritas neste enunciado poderá acarretar penalizações na nota final ou desclassificação da avaliação.
- Não serão aceitas entregas por e-mail, mensagens ou qualquer outro meio de comunicação externo ao Portal Acadêmico.
- Trabalhos com conteúdo idêntico ou excessivamente semelhantes serão zerados, por configurarem plágio ou cópia indevida.

- A entrega é individual. Caso o aluno não realize a submissão no ambiente acadêmico será desconsiderada a nota da entrega.

Atenciosamente,
— Secretaria Labdata.